COLLEGE DE PARIS

TUTORIEL PROJET TUTORÉ

Application de Gestion de Stock

Python, Tkinter & PostgreSQL

Réalisé par : [LAWSON Latevi Sena]

Enseignant : [LAWSON Latevi Sena]

Université : Collège de Paris - Campus de Lomé

Janvier 2026

Table des matières

1 Introduction

1.1 Présentation du projet

Ce tutoriel vous guide dans la création d'une application complète de gestion de stock en Python. Le projet permet de mettre en pratique les concepts fondamentaux de la programmation Python, notamment :

- La Programmation Orientée Objet (POO)
- Les interfaces graphiques avec Tkinter
- La gestion de bases de données avec PostgreSQL
- Les opérations CRUD (Create, Read, Update, Delete)

1.2 Objectifs pédagogiques

À la fin de ce projet, vous serez capable de :

1. Créer une interface graphique intuitive avec Tkinter
2. Connecter une application Python à une base de données PostgreSQL
3. Implémenter les opérations de base sur une base de données
4. Gérer les erreurs et les exceptions
5. Organiser votre code en modules réutilisables

1.3 Prérequis

- Connaissances de base en Python
- Notions de programmation orientée objet
- Compréhension des bases de données relationnelles
- Python 3.7 ou supérieur installé

2 Installation de l'environnement

2.1 Installation de Python

Python 3 doit être installé sur votre système. Vérifiez la version :

```
python3 --version
```

Si Python n'est pas installé :

```
# Ubuntu/Debian
sudo apt update
sudo apt install python3 python3-pip

# macOS (avec Homebrew)
brew install python3
```

2.2 Installation de PostgreSQL

Sur Ubuntu/Debian :

```
sudo apt update
sudo apt install postgresql postgresql-contrib
```

Sur macOS :

```
brew install postgresql
brew services start postgresql
```

Sur Windows :
Téléchargez l'installateur depuis `https://www.postgresql.org/download/windows/`

2.3 Installation de Tkinter

```
# Ubuntu/Debian
sudo apt install python3-tk

# macOS (g n ralement inclus)
# V rifier avec : python3 -m tkinter
```

2.4 Configuration de PostgreSQL

Étape 1 : Démarrer PostgreSQL

```
sudo service postgresql start
```

Étape 2 : Définir un mot de passe pour l'utilisateur postgres

```
sudo -u postgres psql
```

Dans la console PostgreSQL :

```
ALTER USER postgres WITH PASSWORD 'postgres';
\q
```

Étape 3 : Créer la base de données

```
sudo -u postgres psql -c "CREATE DATABASE gestion_stock;"
```

3 Structure du projet

3.1 Organisation des fichiers

Créez un dossier pour votre projet :

```
mkdir GESTIO_DE_STOCK_PYTHON
cd GESTIO_DE_STOCK_PYTHON
```

Le projet sera organisé ainsi :

```
GESTIO_DE_STOCK_PYTHON/
 main.py              # Interface principale
 database.py          # Gestion base de données
 requirements.txt     # Dépendances
 README.md           # Documentation
```

3.2 Création du fichier requirements.txt

Ce fichier liste les dépendances Python nécessaires.

Fichier : requirements.txt

```
psycopg2-binary==2.9.9
```

Installez les dépendances :

```
pip install -r requirements.txt
```

4 Module de gestion de la base de données

4.1 Concepts SQL utilisés

Structure de la table produits :

- `id` : Identifiant unique (clé primaire, auto-incrémenté)
- `nom` : Nom du produit (VARCHAR)
- `categorie` : Catégorie du produit (VARCHAR)
- `quantite` : Quantité en stock (INTEGER)
- `prix` : Prix unitaire (DECIMAL)

4.2 Création du module database.py

Ce module contient une classe `Database` qui encapsule toutes les opérations de base de données.

Fichier : database.py

```
# -*- coding: utf-8 -*-
"""
Module de gestion de la base de donn es PostgreSQL
"""
import psycopg2
from psycopg2 import Error

class Database:
    def __init__(self):
        self.connection = None
        self.cursor = None
```

4.2.1 Méthode de connexion

```
def connecter(self, host="localhost", database="gestion_stock",
              user="postgres", password=""):
    """ tablit  la connexion   PostgreSQL"""
    try:
        self.connection = psycopg2.connect(
            host=host,
            database=database,
            user=user,
            password=password
        )
```

```
        self.cursor = self.connection.cursor()
        print("Connexion    PostgreSQL r ussie")
        return True
    except Error as e:
        print("Erreur de connexion: {}".format(e))
        return False
```

4.2.2 Création de la table

```
def creer_table(self):
    """Cr e la table des produits si elle n'existe pas"""
    try:
        query = """
        CREATE TABLE IF NOT EXISTS produits (
            id SERIAL PRIMARY KEY,
            nom VARCHAR(100) NOT NULL,
            categorie VARCHAR(50),
            quantite INTEGER NOT NULL,
            prix DECIMAL(10, 2) NOT NULL
        )
        """
        self.cursor.execute(query)
        self.connection.commit()
        print("Table cr   e avec succ s")
    except Error as e:
        print("Erreur lors de la cr ation: {}".format(e))
```

4.2.3 Opérations CRUD

Create - Ajouter un produit :

```
def ajouter_produit(self, nom, categorie, quantite, prix):
    """Ajoute un nouveau produit"""
    try:
        query = "INSERT INTO produits (nom, categorie, quantite, prix)
    VALUES (%s, %s, %s, %s)"
        self.cursor.execute(query, (nom, categorie, quantite, prix))
        self.connection.commit()
        return True
    except Error as e:
        print("Erreur d'ajout: {}".format(e))
        return False
```

Read - Lire tous les produits :

```
def obtenir_tous_produits(self):
    """R cup re tous les produits"""
    try:
        query = "SELECT * FROM produits ORDER BY id"
        self.cursor.execute(query)
        return self.cursor.fetchall()
    except Error as e:
        print("Erreur de lecture: {}".format(e))
        return []
```

Update - Modifier un produit :

```
def modifier_produit(self, id_produit, nom, categorie,
                     quantite, prix):
    """Modifie un produit existant"""
    try:
```

```
        query = """UPDATE produits
                   SET nom=%s, categorie=%s, quantite=%s, prix=%s
                   WHERE id=%s"""
        self.cursor.execute(query,
            (nom, categorie, quantite, prix, id_produit))
        self.connection.commit()
        return True
    except Error as e:
        print("Erreur de modification: {}".format(e))
        return False
```

Delete - Supprimer un produit :

```
def supprimer_produit(self, id_produit):
    """Supprime un produit"""
    try:
        query = "DELETE FROM produits WHERE id=%s"
        self.cursor.execute(query, (id_produit,))
        self.connection.commit()
        return True
    except Error as e:
        print("Erreur de suppression: {}".format(e))
        return False
```

Déconnexion :

```
def deconnecter(self):
    """Ferme la connexion"""
    if self.cursor:
        self.cursor.close()
    if self.connection:
        self.connection.close()
        print("Connexion PostgreSQL ferm e")
```

5 Interface graphique avec Tkinter

5.1 Architecture de l'interface

L'application est structurée en deux parties :

1. **Interface de connexion** : Pour se connecter à PostgreSQL
2. **Interface principale** : Pour gérer les produits

5.2 Création du fichier main.py

5.2.1 Initialisation de la classe principale

Fichier : main.py

```
# -*- coding: utf-8 -*-
"""
Application de Gestion de Stock avec Tkinter et PostgreSQL
"""
import tkinter as tk
from tkinter import ttk, messagebox
from database import Database

```

```
class ApplicationGestionStock:
    def __init__(self, root):
        self.root = root
        self.root.title("Gestion de Stock")
        self.root.geometry("900x600")

        # Initialiser la base de donn es
        self.db = Database()

        # Cr er l'interface de connexion
        self.creer_interface_connexion()
```

5.2.2 Interface de connexion

```
def creer_interface_connexion(self):
    """Cr e l'interface de connexion    PostgreSQL"""
    self.frame_connexion = tk.Frame(self.root, padx=20, pady=20)
    self.frame_connexion.pack(expand=True)

    tk.Label(self.frame_connexion,
             text="Connexion    PostgreSQL",
             font=("Arial", 16, "bold")).grid(row=0, column=0,
                                              columnspan=2, pady=10)

    # Champs de saisie
    tk.Label(self.frame_connexion, text="H te:").grid(
        row=1, column=0, sticky="e", padx=5, pady=5)
    self.entry_host = tk.Entry(self.frame_connexion, width=30)
    self.entry_host.insert(0, "localhost")
    self.entry_host.grid(row=1, column=1, pady=5)

    # ... (autres champs similaires)

    # Bouton de connexion
    tk.Button(self.frame_connexion, text="Se connecter",
              command=self.connecter_db,
              bg="#4CAF50", fg="white").grid(
                  row=5, column=0, columnspan=2, pady=20)
```

5.2.3 Méthode de connexion

```
def connecter_db(self):
    """Connecte    la base de donn es"""
    host = self.entry_host.get()
    database = self.entry_database.get()
    user = self.entry_user.get()
    password = self.entry_password.get()

    if self.db.connecter(host, database, user, password):
        self.db.creer_table()
        self.frame_connexion.destroy()
        self.creer_interface_principale()
    else:
        messagebox.showerror("Erreur",
            "Impossible de se connecter    la base de donn es")
```

5.3 Interface principale

5.3.1 Formulaire de saisie

```
def creer_interface_principale(self):
    """Cr e l'interface principale"""
    # Frame pour le formulaire
    frame_formulaire = tk.LabelFrame(
        self.root,
        text="Ajouter/Modifier un produit",
        padx=10, pady=10)
    frame_formulaire.pack(fill="x", padx=10, pady=10)

    # Champs du formulaire
    tk.Label(frame_formulaire,
             text="Nom du produit:").grid(
                 row=0, column=0, sticky="w", padx=5, pady=5)
    self.entry_nom = tk.Entry(frame_formulaire, width=30)
    self.entry_nom.grid(row=0, column=1, padx=5, pady=5)

    # ... (autres champs)
```

5.3.2 Boutons d'action

```
    # Boutons d'action
    frame_boutons = tk.Frame(frame_formulaire)
    frame_boutons.grid(row=2, column=0, columnspan=4, pady=10)

    tk.Button(frame_boutons, text="Ajouter",
              command=self.ajouter_produit,
              bg="#4CAF50", fg="white",
              width=15).pack(side="left", padx=5)
    tk.Button(frame_boutons, text="Modifier",
              command=self.modifier_produit,
              bg="#2196F3", fg="white",
              width=15).pack(side="left", padx=5)
    tk.Button(frame_boutons, text="Supprimer",
              command=self.supprimer_produit,
              bg="#f44336", fg="white",
              width=15).pack(side="left", padx=5)
```

5.3.3 Tableau des produits (Treeview)

```
    # Frame pour le tableau
    frame_tableau = tk.LabelFrame(
        self.root,
        text="Liste des produits",
        padx=10, pady=10)
    frame_tableau.pack(fill="both", expand=True,
                       padx=10, pady=10)

    # Scrollbar
    scrollbar = tk.Scrollbar(frame_tableau)
    scrollbar.pack(side="right", fill="y")

    # Treeview
    self.tree = ttk.Treeview(
        frame_tableau,
        columns=("ID", "Nom", "Cat gorie", "Quantit ", "Prix"),
```

```
        show="headings",
        yscrollcommand=scrollbar.set)
    scrollbar.config(command=self.tree.yview)

    # D finir les en-t tes
    self.tree.heading("ID", text="ID")
    self.tree.heading("Nom", text="Nom du produit")
    self.tree.heading("Cat gorie", text="Cat gorie")
    self.tree.heading("Quantit ", text="Quantit ")
    self.tree.heading("Prix", text="Prix (   )")

    # Largeur des colonnes
    self.tree.column("ID", width=50, anchor="center")
    self.tree.column("Nom", width=200)
    self.tree.column("Cat gorie", width=150)
    self.tree.column("Quantit ", width=100, anchor="center")
    self.tree.column("Prix", width=100, anchor="center")

    self.tree.pack(fill="both", expand=True)

    # Bind pour s lection
    self.tree.bind("<ButtonRelease-1>",
                   self.selectionner_produit)

    # Charger les produits
    self.actualiser_tableau()
```

5.4 Méthodes d'action

5.4.1 Ajouter un produit

```
def ajouter_produit(self):
    """Ajoute un nouveau produit"""
    nom = self.entry_nom.get()
    categorie = self.entry_categorie.get()
    quantite = self.entry_quantite.get()
    prix = self.entry_prix.get()

    if not nom or not quantite or not prix:
        messagebox.showwarning("Attention",
            "Veuillez remplir tous les champs obligatoires")
        return

    try:
        quantite = int(quantite)
        prix = float(prix)

        if self.db.ajouter_produit(nom, categorie,
                                   quantite, prix):
            messagebox.showinfo("Succ s",
                "Produit ajout  avec succ s")
            self.effacer_champs()
            self.actualiser_tableau()
        else:
            messagebox.showerror("Erreur",
                "Impossible d'ajouter le produit")
    except ValueError:
        messagebox.showerror("Erreur",
```

```
            "Quantit  et prix doivent  tre  des nombres")
```

5.4.2 Modifier un produit

```
def modifier_produit(self):
    """Modifie le produit s lectionn """
    selection = self.tree.selection()
    if not selection:
        messagebox.showwarning("Attention",
            "Veuillez s lectionner un produit    modifier")
        return

    item = self.tree.item(selection[0])
    id_produit = item['values'][0]

    nom = self.entry_nom.get()
    categorie = self.entry_categorie.get()
    quantite = self.entry_quantite.get()
    prix = self.entry_prix.get()

    try:
        quantite = int(quantite)
        prix = float(prix)

        if self.db.modifier_produit(id_produit, nom,
                                    categorie, quantite, prix):
            messagebox.showinfo("Succ s",
                "Produit modifi  avec succ s")
            self.effacer_champs()
            self.actualiser_tableau()
    except ValueError:
        messagebox.showerror("Erreur",
            "Quantit  et prix doivent  tre  des nombres")
```

5.4.3 Supprimer un produit

```
def supprimer_produit(self):
    """Supprime le produit s lectionn """
    selection = self.tree.selection()
    if not selection:
        messagebox.showwarning("Attention",
            "Veuillez s lectionner un produit    supprimer")
        return

    item = self.tree.item(selection[0])
    id_produit = item['values'][0]
    nom_produit = item['values'][1]

    reponse = messagebox.askyesno("Confirmation",
        "Voulez-vous vraiment supprimer '{}'?".format(
            nom_produit))
    if reponse:
        if self.db.supprimer_produit(id_produit):
            messagebox.showinfo("Succ s",
                "Produit supprim  avec succ s")
            self.effacer_champs()
            self.actualiser_tableau()
```

5.4.4 Méthodes utilitaires

```
def selectionner_produit(self, event):
    """Remplit les champs avec le produit s lectionn """
    selection = self.tree.selection()
    if selection:
        item = self.tree.item(selection[0])
        values = item['values']

        self.effacer_champs()
        self.entry_nom.insert(0, values[1])
        self.entry_categorie.insert(0, values[2])
        self.entry_quantite.insert(0, values[3])
        self.entry_prix.insert(0, values[4])

def effacer_champs(self):
    """Efface tous les champs du formulaire"""
    self.entry_nom.delete(0, tk.END)
    self.entry_categorie.delete(0, tk.END)
    self.entry_quantite.delete(0, tk.END)
    self.entry_prix.delete(0, tk.END)

def actualiser_tableau(self):
    """Actualise le tableau avec les donn es"""
    # Effacer le tableau
    for item in self.tree.get_children():
        self.tree.delete(item)

    # Charger les produits
    produits = self.db.obtenir_tous_produits()
    for produit in produits:
        self.tree.insert("", "end", values=produit)

def fermer_application(self):
    """Ferme l'application proprement"""
    self.db.deconnecter()
    self.root.destroy()
```

5.5 Point d'entrée de l'application

```
if __name__ == "__main__":
    root = tk.Tk()
    app = ApplicationGestionStock(root)
    root.protocol("WM_DELETE_WINDOW", app.fermer_application)
    root.mainloop()
```

6 Lancement de l'application

6.1 Vérification de l'installation

Assurez-vous que :

1. PostgreSQL est démarré
2. La base de données `gestion_stock` existe
3. Les dépendances Python sont installées

6.2 Exécution

Dans le terminal, depuis le dossier du projet :

```
python3 main.py
```

6.3 Utilisation de l'application

6.3.1 Première connexion

1. Remplissez les informations de connexion
2. Cliquez sur "Se connecter"
3. L'interface principale s'affiche

6.3.2 Gestion des produits

Ajouter un produit :

1. Remplissez les champs du formulaire
2. Cliquez sur "Ajouter"
3. Le produit apparaît dans le tableau

Modifier un produit :

1. Sélectionnez un produit dans le tableau
2. Modifiez les champs souhaités
3. Cliquez sur "Modifier"

Supprimer un produit :

1. Sélectionnez un produit dans le tableau
2. Cliquez sur "Supprimer"
3. Confirmez la suppression

7 Concepts Python appliqués

7.1 Programmation Orientée Objet

POO

— **Classes** : `Database`, `ApplicationGestionStock`
— **Méthodes** : Fonctions membres des classes
— **Attributs** : Variables d'instance (`self.connection`, `self.root`)
— **Encapsulation** : Données et méthodes regroupées

7.2 Gestion des exceptions

Try/Except

Utilisation de `try/except` pour :

- Gérer les erreurs de connexion
- Valider les entrées utilisateur
- Attraper les erreurs SQL

7.3 Modules Python

Modules

- `tkinter` : Interface graphique
- `psycopg2` : Connexion PostgreSQL
- Modules personnalisés : `database.py`

7.4 Opérations CRUD

CRUD

- **Create** : INSERT INTO
- **Read** : SELECT
- **Update** : UPDATE SET
- **Delete** : DELETE FROM

8 Dépannage

8.1 Problèmes courants

8.1.1 Erreur de connexion PostgreSQL

Symptôme : ”Impossible de se connecter à la base de données”
Solutions :

- Vérifier que PostgreSQL est démarré : `sudo service postgresql status`
- Vérifier le mot de passe
- Vérifier que la base de données existe

8.1.2 Module psycopg2 non trouvé

Symptôme : ”ModuleNotFoundError : No module named ’psycopg2’”
Solution :

```
pip install psycopg2-binary
```

8.1.3 Module tkinter non trouvé

Symptôme : ”ModuleNotFoundError : No module named ’tkinter’”
Solution :

```
sudo apt install python3-tk
```

8.1.4 Erreur de syntaxe avec les caractères français

Symptôme : ”SyntaxError : Non-ASCII character”
Solution : Ajouter en première ligne :

```
# -*- coding: utf-8 -*-
```

9 Améliorations possibles

9.1 Fonctionnalités supplémentaires

1. **Recherche et filtrage**
   - Ajouter une barre de recherche
   - Filtrer par catégorie
2. **Alertes de stock**
   - Afficher un avertissement si quantité ¡ seuil
   - Calculer la valeur totale du stock
3. **Export de données**
   - Exporter en CSV
   - Générer des rapports PDF
4. **Statistiques**
   - Graphiques avec matplotlib
   - Analyse des tendances
5. **Authentification**
   - Gestion des utilisateurs
   - Droits d’accès

9.2 Exercices pratiques

1. Ajouter un champ ”Date d’ajout” dans la table
2. Créer une fonction de recherche par nom
3. Ajouter un bouton ”Exporter en CSV”
4. Implémenter une validation des données avant insertion
5. Créer une fenêtre de statistiques

10 Conclusion

Ce projet vous a permis de :

— Créer une application complète avec interface graphique
— Maîtriser les bases de données relationnelles
— Appliquer les concepts de la POO
— Gérer les erreurs et les exceptions
— Organiser un projet Python professionnel

10.1 Ressources complémentaires

— Documentation Python : `https://docs.python.org/fr/3/`
— Documentation Tkinter : `https://docs.python.org/fr/3/library/tkinter.html`
— Documentation PostgreSQL : `https://www.postgresql.org/docs/`
— Documentation psycopg2 : `https://www.psycopg.org/docs/`

Bon courage pour votre projet tutoré !